Kinect 1

Francesco Angelo Fabiano Antonacci, Marco Laccu, and Giordano Notarnicola
Università Pisa
(Dated: 21 maggio 2026)

In questo articolo si espongono le caratteristiche trovate sperimentalmente e il funzionamento di Kinect v1 Microsoft 2010. Si mostrano alcune applicazioni volte a mostrarne i limiti.

HARDWARE

Il Kinect v1 2010 Microsoft (Fig.(1)) è dotato di una telecamera RGB con risoluzione di 640×480 pixel e campo visivo di $63° \times 50°$, di un laser a 830 nm che proietta una griglia di *speckles* e di una telecamera sensibile alla stessa banda infrarossa, anch'essa con risoluzione di 640×480 pixel e campo visivo di $57° \times 47°$.



Figura 1: Kinect v1 2010 Microsoft

La combinazione delle due telecamere consente di acquisire una visione tridimensionale dell'ambiente circostante mediante una misura di profondità. L'acquisizione avviene a 30 fps. Il sensore è inoltre dotato di una serie di microfoni e di accelerometri, che tuttavia non sono mai stati utilizzati nel corso degli esperimenti.

Il sensore restituisce tre immagini:

- immagine RGB 640×480 pixel con 8 bit ciascuno;
- immagine IR 640×480 pixel con 11 bit ciascuno;
- immagine delle profondità 640×480 pixel con 11 bit ciascuno.

OBIETTIVI SPERIMENTALI

Sono stati fissati i seguenti obbiettivi sperimentali:

- calibrare le acquisizioni con Kinect;
- studiare le proprietà ottiche;
- studiare i limiti di acquisizione dello strumento;
- eseguire esperimenti di meccanica;
- fare una prova di computer vision.

CONCETTO DI DISPARITÀ

Il Kinect v1 acquisisce le misure di profondità tramite un sistema di triangolazione. Il laser emette un fascio che viene diviso tramite una griglia per diffrazione, proiettando gli *speckles* sulla scena. Il "pattern" è catturato dalla telecamera IR e confrontato con un "pattern" di riferimento nella memoria del dispositivo, che consiste nell'acquisizione di un piano a distanza fissata. Dunque se un oggetto si trova prima o dopo il piano di riferimento, la posizione degli *speckles* risulterà traslata. La traslazione è misurata per ogni speckles e tramite un processo di correlazione, se ne ricava la disparità d. In Fig.(2), viene spiegato come, a partire dalla disparità, il sensore riesce a calcolare la distanza in profondità dell'oggetto. Tramite considerazioni per triangoli simili si ottengono due relazioni:

$$\frac{D}{b} = \frac{Z_0 - Z_k}{Z_0} \quad \text{e} \quad \frac{d}{f} = \frac{D}{Z_k}$$

Unendo le due equazioni, si trova una relazione fra la disparità e la posizione in profondità dell'oggetto. Il risultato viene salvato in bit, Nei seguenti paragrafi viene usato il medesimo modello per ottenere una conversione in unità naturali.

$$Z_k = \frac{Z_0}{1 + \frac{Z_0}{bf} d} \tag{1}$$

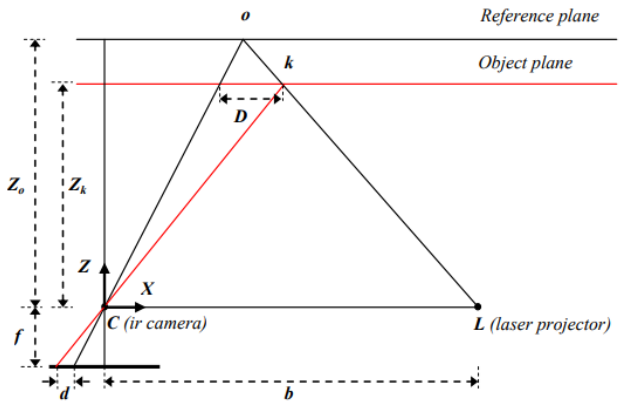


Figura 2: Schema della disparità.

MODELLO PINHOLE

Il modello della camera *pinhole* (o camera stenopeica) rappresenta la base geometrica per comprendere come un sensore digitale catturi il mondo tridimensionale. Per semplificare l'analisi matematica ed evitare di gestire immagini capovolte, si adotta convenzionalmente il modello del **piano immagine virtuale**, posizionato frontalmente rispetto al centro ottico a una distanza pari alla lunghezza focale f (si veda Fig. 2).

Per descrivere la proiezione, si definiscono due sistemi di riferimento:

- **Spazio 3D (Coordinate della fotocamera):** Un punto generico è definito dalla terna (X, Y, Z). L'origine $(0, 0, 0)$ coincide con il **Centro Ottico**. L'asse Z rappresenta l'asse ottico e identifica la profondità.
- **Spazio 2D (Coordinate dell'immagine):** La proiezione sul piano fisico del sensore ha coordinate (x, y).

Sfruttando la geometria dei **triangoli simili** che si formano tra il punto nello spazio, il centro ottico e la proiezione sul piano, possiamo stabilire una relazione di proporzionalità diretta.

Considerando il profilo verticale (piano $Y - Z$):

$$\frac{y}{f} = \frac{Y}{Z} \implies y = f \cdot \frac{Y}{Z} \tag{2}$$

Analogamente, per l'asse orizzontale (piano $X - Z$):

$$x = f \cdot \frac{X}{Z} \tag{3}$$

Queste equazioni definiscono la posizione fisica del punto sul sensore espressa in unità metriche.

Nelle fotocamere digitali, le coordinate devono essere convertite in pixel (u, v). Questa trasformazione richiede l'introduzione dei **parametri intrinseci**:

Poiché i pixel possono non essere perfettamente quadrati, si definisce la lunghezza focale scalata per le dimensioni dei pixel su entrambi gli assi:

- f_x: Focale in pixel lungo l'asse orizzontale.
- f_y: Focale in pixel lungo l'asse verticale.

In un'immagine digitale, l'origine $(0, 0)$ è situata nell'angolo in alto a sinistra. Si deve quindi traslare il risultato aggiungendo le coordinate del **punto principale** (c_x, c_y), che rappresenta l'intersezione dell'asse ottico con il piano immagine.

Si ottiene così il modello *pinhole* completo:

$$u = f_x \cdot \frac{X}{Z} + c_x \tag{4}$$

$$v = f_y \cdot \frac{Y}{Z} + c_y \tag{5}$$

Invertendo le equazioni precedenti si ottiene:

$$X = Z \cdot \frac{(u - c_x)}{f_x} \tag{6}$$

$$Y = Z \cdot \frac{(v - c_y)}{f_y} \tag{7}$$

CALIBRAZIONI LUNGO GLI ASSI

Calibrazione lungo x-y Si sono scattate con il kinect 2 fotografie, una in formato RGB e in infrarosso, di una griglia formata da quadrati in alluminio (i metalli riflettono bene la radiazione) con passo 30 *cm*, posta su un muro a 140 *cm* di distanza dal sensore. Si è poi estratta la posizione dei pixel di alcuni punti.

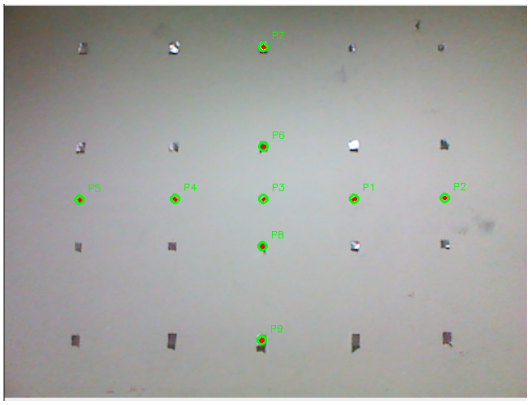


Figura 3: Procedura per la calibrazione della profondità.

Conoscendo la posizione reale dei punti nel piano (X,Y) e la sua posizione in pixel, si ottiene:

$$X = Z \cdot \frac{(u - c_x)}{f_x} \qquad Y = Z \cdot \frac{(v - c_y)}{f_y}.$$

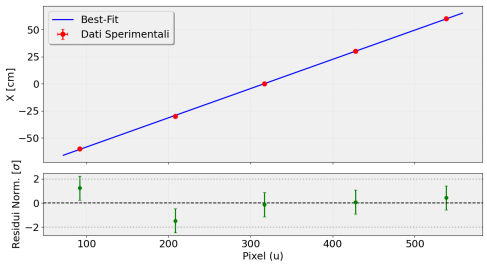


Figura 4: Calibrazione asse X per fotocamera RGB.

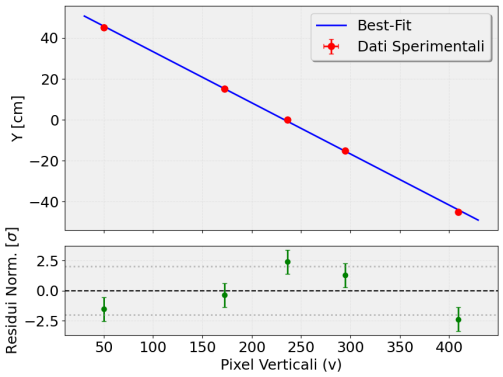


Figura 5: Calibrazione asse Y per fotocamera RGB.

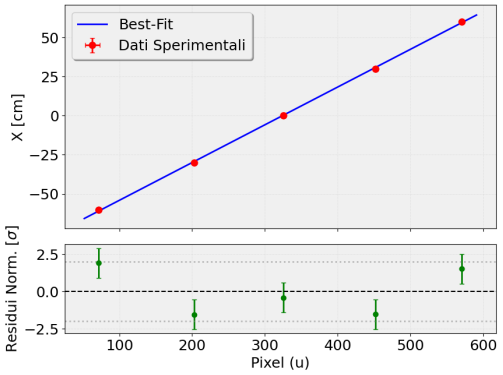


Figura 6: Calibrazione asse X per fotocamera IR.

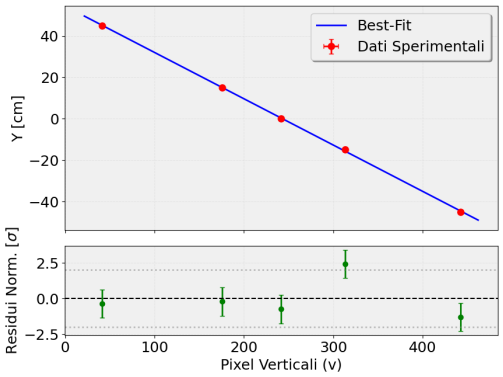


Figura 7: Calibrazione asse Y per fotocamera IR.

Calibrazione lungo z Si è preso un corridoio ed è stato misurato un bersaglio a diverse distanze note come mostrato in Fig.(8). Per prendere le acquisizioni in diversi momenti è stato utile allineare il bersaglio con un riferimento in fondo al corridoio.

Il kinect restituisce valori in bit per le misure di profondità che possono essere convertite in metri con:

$$D(x) = \frac{\beta_1}{\beta_0 - x} \tag{8}$$

Tabella I: Parametri di calibrazione ottenuti tramite fit per le fotocamere RGB e IR ($Z = 140.0$ cm).

Parametro	**Fotocamera RGB**	**Fotocamera IR**
Focale f_x [px]	519.0 ± 3.3	581 ± 5
Centro ottico c_x [px]	317 ± 1	324.4 ± 1.5
χ^2_ν (X)	1.30	3.64
Focale f_y [px]	560.9 ± 7.3	625.4 ± 1.1
Centro ottico c_y [px]	$233. \pm 1.4$	243.0 ± 1.1
χ^2_ν (Y)	5.14	3.87

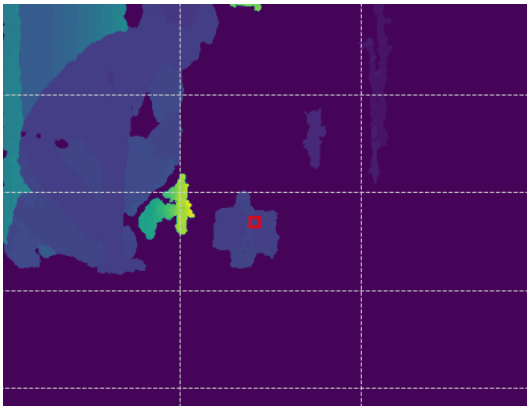


Figura 8: Procedura per la calibrazione della profondità.

dove D è la distanza lungo z (asse ortogonale al piano delle telecamere del kinect). È stato dunque eseguito un bestfit per ottenere i parametri del modello in Fig.(9).

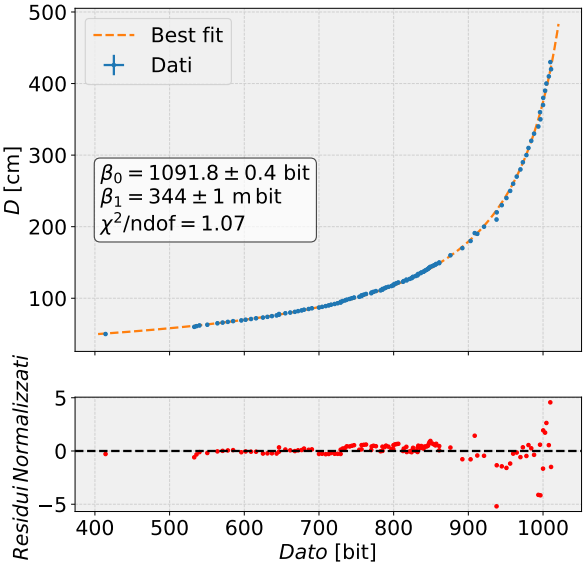


Figura 9: Bestfit per la calibrazione sulle profondità

SOVRAPPOSIZIONE IMMAGINE IR-DEPTH

Il processo si è articolato nelle seguenti fasi:

1. **Scatto delle foto:** sono state scattate le foto in Fig.(10) e Fig.(11), una in formato IR e l'altra depth, di 2 fogli di carta rettangolari appesi ad un supporto.
2. **Pre-elaborazione:** I dati **depth** sono stati normalizzati in un range $0 - 255$ per permetterne la visualizzazione a video.
3. **Selezione:** Sono stati selezionati manualmente due rettangoli corrispondenti agli stessi fogli in entrambe le immagini tramite l'interfaccia OpenCV.
4. **Calcolo dei Centri:** Per ogni rettangolo i, è stato calcolato il centro geometrico (x_c, y_c).
5. **Calcolo della Traslazione:** Si è calcolato la traslazione media e l'incertezza associata

Nella tabella seguente sono riportati i valori ottenuti:

Parametro	**Valore Medio [px]**	**Incertezza [px]**
Trasl. Orizz. (Δx)	4	$\pm$ 1
Trasl. Vert. (Δy)	2	$\pm$ 1

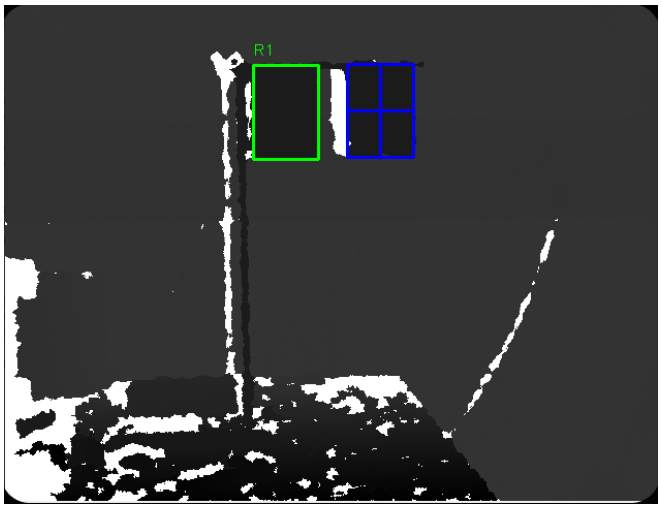


Figura 10: Bestfit per la calibrazione sulle profondità

RISOLUZIONE E LIMITI DELLE FOTOCAMERE

Dalle calibrazioni eseguite è stato possibile ricavare le risoluzioni spaziali del kinect, come si può vedere in Fig.(12).

Sebbene, minore è la distanza dalle fotocamere, maggiore è la precisione delle acquisizioni, il kinect non è

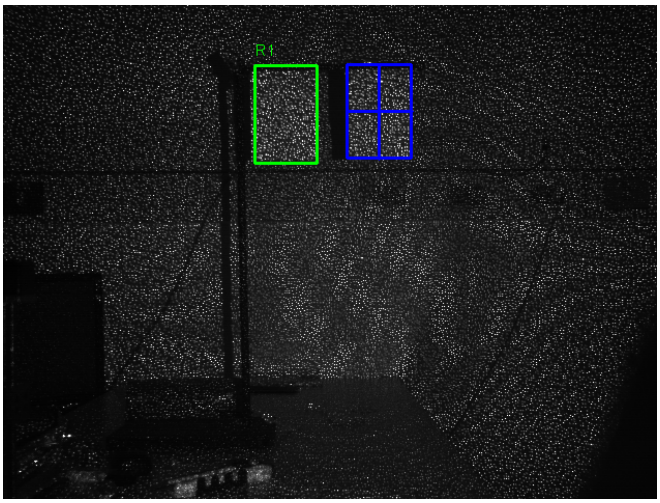


Figura 11: Bestfit per la calibrazione sulle profondità

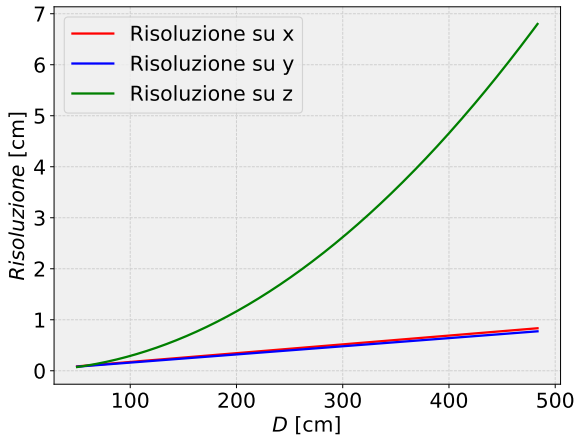


Figura 12: Risoluzioni spaziali del Kinect. Notare: per i diversi principi fisici utilizzati nella acquisizioni, quelle su x e y crescono linearmente con la distanza, mentre quella su z cresce quadraticamente.

capace di fare misure di profondità a distanze inferiori ai 50 cm e consistentemente ai 60 cm.

Il motivo di ciò il fatto che è necessario dello spazio affinchè geometricamente gli speckles possano essere visti dalla fotocamera IR e non essere bloccati prima: senza di essi la fotocamera IR riesce comunque ad acquisire dati ma kinect non riesce a fare misure di profondità.

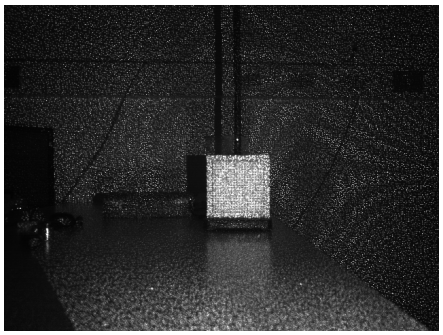
Per un motivo simile accade che nelle acquisizioni si vedano delle ombre di oggetti, come in figura 14 nella scena che oscura gli oggetti retrostanti: queste sono sempre dovute alla non coincidenza spaziale tra il proiettore e la fotocamera.

Per ovviare a questo problema la soluzione più facile è sacrificare un po' di risoluzione ed allontanarsi dalla fotocamera.

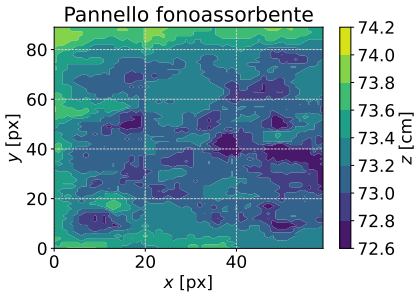
Un ulteriore limite alla risoluzione spaziale è legato alla regolarità della superficie: per la tecnica con la quale Kinect v1 stima le distanze, superfici frastagliate comunque entro i limiti della risoluzione non riescono ad essere ricostruite come si può vedere in Fig.(13).



(a) Le scanalature sono visibili nella foto RGB.



(b) Le scanalature sono visibili nella foto IR. Tuttavia si è al limite della sensibilità su x e y di Kinect.



(c) Si è ingrandita la sezione dell'immagine col pannello fonoassorbente e, nonostante kinect riesca a stimare bene la distanza dalle fotocamere, non riesce ad osservare le scanalature.

Figura 13: Si è messo questo pannello fonoassorbente alla minima distanza per la quale Kinect riesce a misurare le profondità. Le scanalature del pannello sono di 1 cm, dunque a tale distanza Kinect dovrebbe riuscire a vederle. Si ringrazia Alessia Di Nino per avercelo gentilmente prestato.

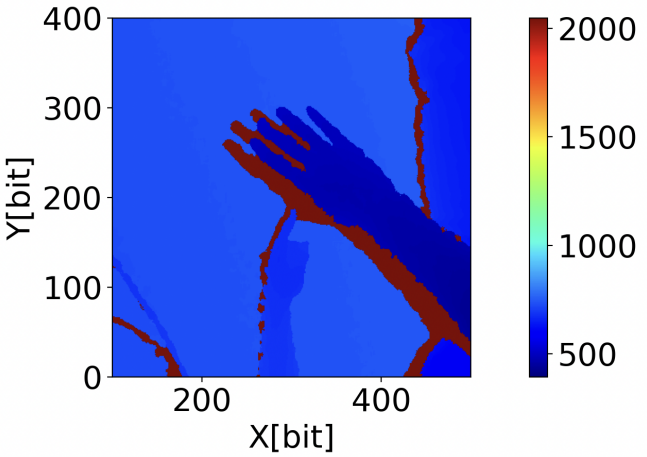


Figura 14: Acquisizione in modalità *depth* di una mano: gli speckles bloccati formano un'ombra nella telecamera IR, restituendo un risultato scalibrato.

È stata fatta un'acquisizione delle profondità di una parete ed è stato fatto un bestfit planare, si veda Fig.(15). Ammesso, e forse non concesso, che il muro sia piatto, dai residui emergono possibili distorsioni della lente ad infrarosso.

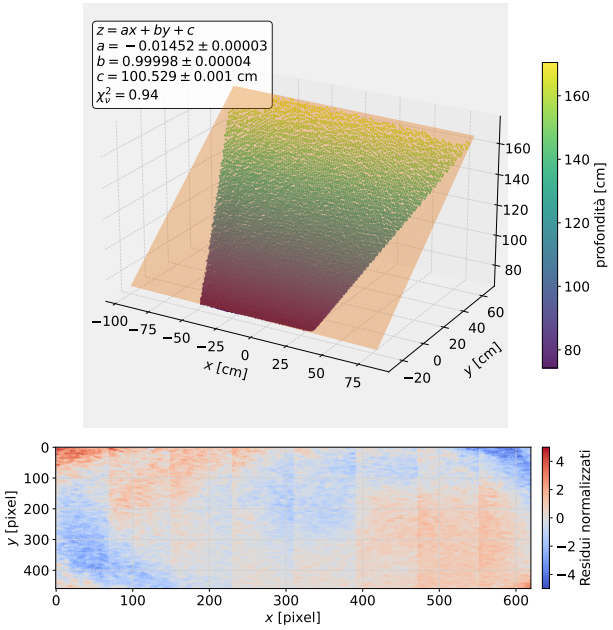


Figura 15: Bestfit planare per una parete.

Inoltre, sia la fotocamera RGB che quella a IR sono mandate in saturazione dalla luce solare: non è possibile fare alcun tipo di acquisizione di giorno all'aperto.

Temporalmente la risoluzione è di 30 Hz e costituisce il vero limite negli studi di meccanica possibili col kinect.

ESPERIMENTI DI DINAMICA

Introduzione

Gli esperimenti che meglio si prestano all'utilizzo di una telecamera RGB sono gli esperimenti di dinamica, nei quali a partire dalla traiettoria di un oggetto si può risalire ai parametri che descrivono l'apparato sperimentale. La possibilità di usare la modalità *depth* semplifica ulteriormente il lavoro perché con un semplice filtro permette di acquisire istantaneamente il dato da centinaia di immagini senza doverle analizzare una alla volta. Una volta acquisiti i frame si passa per un analisi preliminare in cui si "incornicia" la regione del moto su x-y, per poi selezionare un range verticale di punti in cui potrebbe trovarsi l'oggetto. Gli esperimenti successivi includono tutti il tracciamento di una sfera, dunque si è pensato ad un filtro *ad hoc* che seleziona il centro geometrico dell'oggetto e le salva in un file di testo, pronte per l'analisi. Le analisi risultano notevolmente semplificate se viene lasciato spazio libero attorno alla regione di moto: si va a creare così un'immagine ad alto contrasto nella qual il centro di massa della pallina si trova come centro di una maschera che la isola rispetto allo sfondo, si veda Fig.(16).

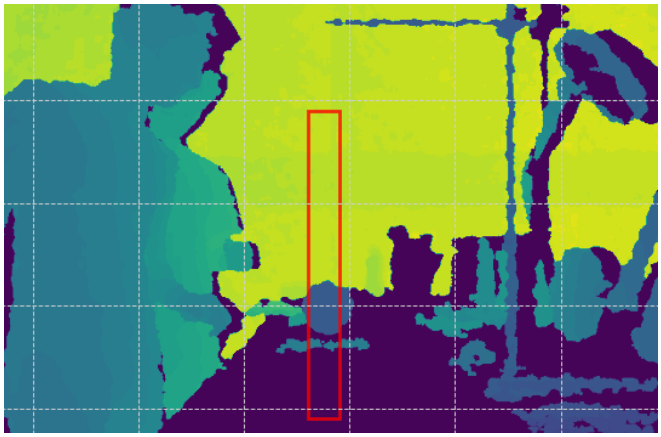


Figura 16: Esempio di acquisizione per un esperimento di dinamica. La pallina è in alto contrasto rispetto allo sfondo, si applica una maschera che con un po' di tolleranza la seleziona e si limita l'analisi e il calcolo del centro delle coordinate solo ad una regione di interesse (qui il rettangolo rosso).

Rimbalzi di una palla

Si studia il moto di un pallina che rimbalza sul pavimento. Dalle leggi della cinematica si ha che

$$h = \frac{v^2}{2g}.$$

A questo modello bisogna aggiungere la dissipazione di energia per urto parzialmente anelastico col pavimento, preso $\gamma < 1$ con

$$E_{n+1} = \gamma E_n$$

che si può applicare ad una legge sulle altezze

$$h_{n+1} = \gamma^n h_0.$$

Si vuole confrontare il moto di una pallina da ping pong e una da tennis, studiando la differenza tra i coefficienti γ si può fare un'analisi qualitativa sui materiali di cui sono costituite.

Con il filtro citato nella sezione *Introduzione* negli *Esperimenti di dinamica* si acquisiscono i dati dei rimbalzi di entrambe le palline e vengono eseguiti fit parabolici per determinarne l'altezza massima per ogni rimbalzo. Viene eseguito un ulteriore fit che restituisce i coefficienti di perdita di energia in Fig.(17) e Fig.(19). In questo esperimento non c'è la vera necessità di convertire i pixel in unità naturali perché γ è adimensionale. I residui evidenziano correlazioni, dunque si potrebbero avere dei sistematici. Il modello potrebbe essere parziale (si sta ignorando l'attrito con l'aria) oppure c'è un errore nel sistema di tracking. In Fig.(18) si riportano i residui dello stesso fit per una pallina da ping pong diversa trovati nel corso di Lab1: è ragionevole ci sia o un problema nel tracking o che la variazione nelle condizioni iniziali e della pallina determino i diversi esiti dell'esperimento. Nel caso della pallina da tennis si hanno a disposizione meno punti in quanto rimbalza veramente poco. Il risultato può essere confrontato con quanto fatto a lab1 sempre con una pallina da tennis, i dati venivano acquisiti con il microfono del computer. I coefficienti γ sono circa gli stessi, però la strumentazione permetteva la raccolta di un maggior numero di punti per la buona risoluzione del microfono, mentre i 30 fps della telecamera IR non sono sufficienti ad una acquisizione così precisa.

Oscillazioni smorzate con una molla

Viene appesa una pallina da tennis ad una molla per studiarne il moto armomico smorzato. Anche in questo caso non viene fatta alcuna conversione da bit in metri perchè i parametri di fit interessanti non dipendono dalle lunghezze. I risultati sono riportati in figura (21).

Pendolo sferico con molla

Successivamente è stato fatto lo studio con la stessa tecnica ma per un pendolo sferico attaccato ad una molla, si riportano i risultati di due acquisizioni in figura (22).

Piccole variazioni nelle condizioni iniziali comportano traiettorie totalmente diverse.

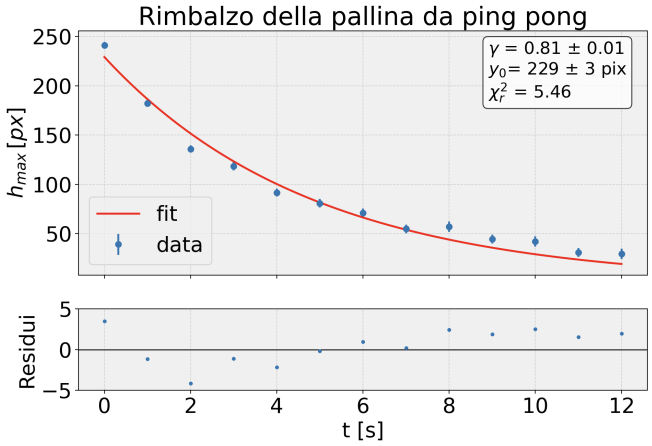


Figura 17: Fit esponenziale delle altezze massime di una pallina da ping pong rispetto al numero di rimbalsi.

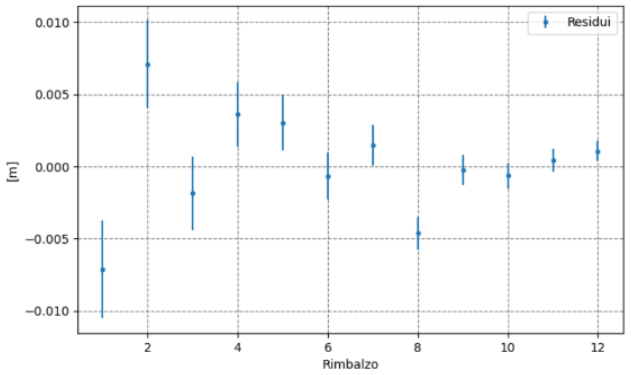


Figura 18: Residui dei rimbalzi di una pallina da ping pong per Laboratorio 1. Non si osserva una particolare correlazione dei residui. Venne stato stimato $\gamma = 0.784 \pm 0.008$.

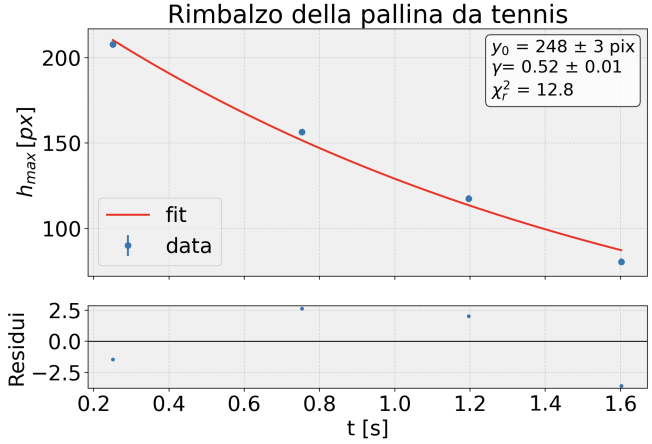


Figura 19: Fit esponenziale eseguito a Lab 1.

E' stato scritto un integratore Euleriano iterando le

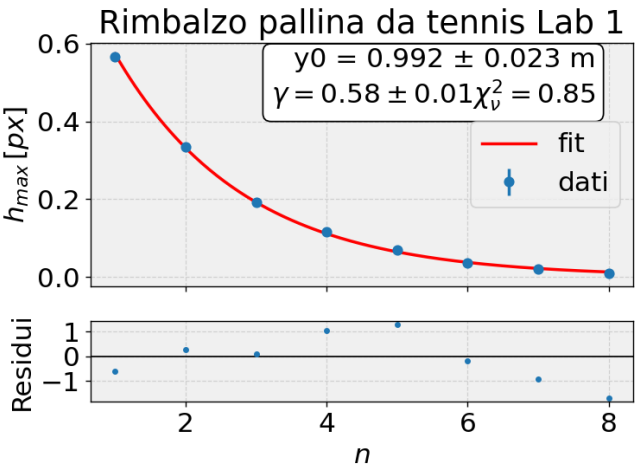


Figura 20: Fit esponenziale eseguito a Lab 1 per pallina da tennis che rimbalza.

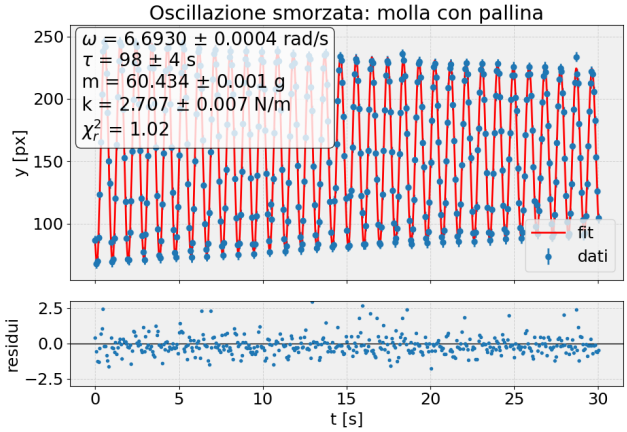


Figura 21: Dati e best fit delle oscillazioni di una pallina attaccata con una molla.

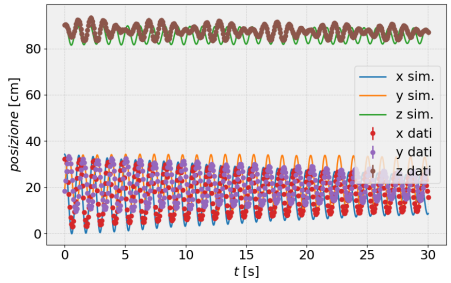
Eq.(9)

$$
\begin{aligned}
\vec{l_n} &= \vec{x_n} - \vec{x_0};\\
\vec{a_n} &= (k/m)(||\vec{l_{n-1}}|| - l_0)\hat{l_{n-1}} - (\gamma/m)\vec{v_{n-1}} - g\hat{z};\\
\vec{x_n} &= \frac{1}{2}\vec{a_n}dt^2 + \vec{v_{n-1}}dt + \vec{x_{n-1}};\\
\vec{v_n} &= \vec{a_n}dt + \vec{v_{n-1}}.
\end{aligned}
\tag{9}
$$

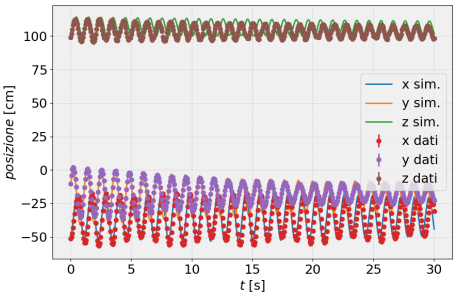
Si ignorano vibrazioni che possono assorbire energia della pallina rispetto alla molla. Si osserva che per $l_0 \neq 0$ e $k \to \infty$ si ha un pendolo sferico semplice. Mentre per $l_0 = 0$ si ha un comportamento da oscillatore armonico.

Il moto che segue talvolta mostra l'accoppiamento tra questi due moti, come si può vedere in Fig.(23).

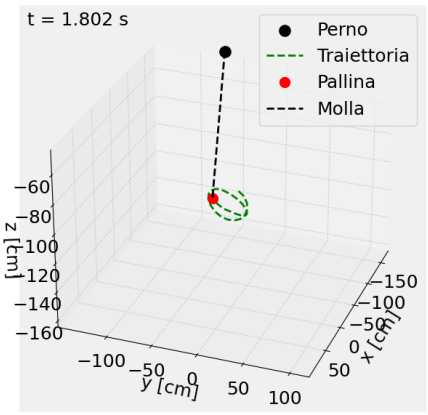
È stato fissato il perno del pendolo con la colla a caldo e ha visivamente ridotto le oscillazioni di questo. È stato provato a fare un'acquisizione e un tentativo di bestfit



(a) Prima acquisizione delle coordinate del pendolo sferico con molla. E' interessante la comparsa di battimenti sull'asse z e le leggi orarie sugli assi che sono sinusoidi deformate.



(b) Seconda acquisizione delle coordinate del pendolo sferico con molla.



(c) Traiettoria dei primi secondi della seconda acquisizione. È stata fatta una traslazione sull'asse z.

Figura 22: Sovrapposta alle due acquisizioni sono stati messi i risultati dell'integratore numerico con i parametri misurati in laboratorio. Il bestfit che si fa da essi non è ragionevole.

riducendosi a un problema 2D, tuttavia non è ancora stato possibile far funzionare il modello.

Si è osservato che facendo oscillare il pendolo su un piano, spontaneamente esso presenta una precessione non

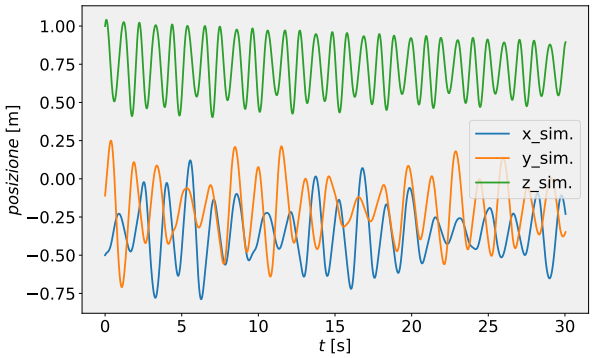


Figura 23: Simulazione del pendolo sferico con condizioni iniziali arbitrarie.

attesa che lo porta a stabilizzarsi su un altro, come si può vedere in Fig.(27). Nell'esperimento realizzato c'è dunque un piano di oscillazione privilegiato legato alla forma del vincolo: questa è, con ogni probabilità, la causa dell'oscillazione sul piano sbagliato.

Si è fatto un'ultima acquisizione più lunga e con ulteriori accortezze, si riporta la traiettoria in Fig.(24) Si ve-

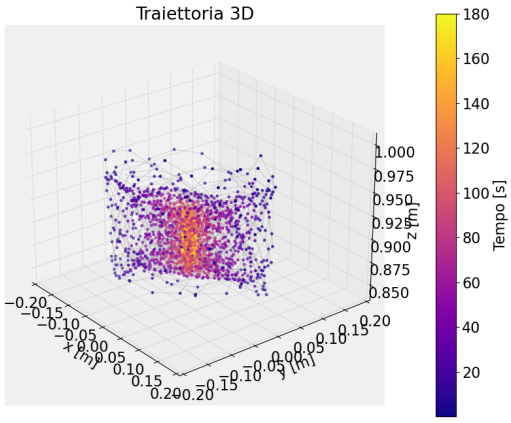


Figura 24: Traiettoria del pendolo sferico com molla in un'acquisizione da 3 min.

de dalle funzioni di autocorrelazioni in Fig.(25) e Fig.(26) che non sono compatibili con quelle di oscillatori armonici smorzati.

E' interessante vedere le traiettorie nello spazio delle fasi come in Fig.(28), Fig.(29) e Fig.(30).

ESPERIMENTI DI FLUIDODINAMICA

L'idea iniziale consisteva nel ricostruire le onde di superficie con un fit tramite le integrazioni delle Navier-

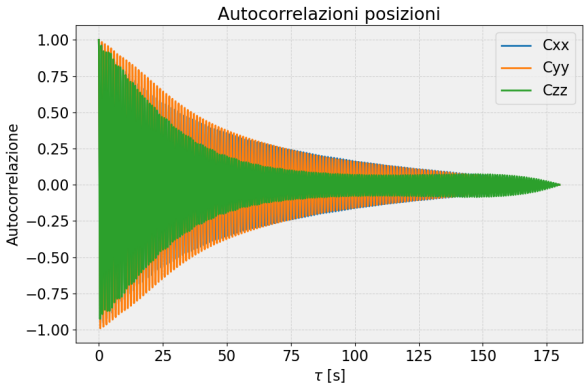


Figura 25: Funzioni di autocorrelazione per le posizioni.

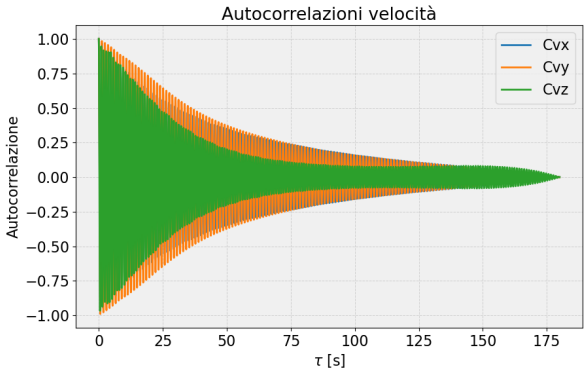


Figura 26: Funzioni di autocorrelazione per le velocità.

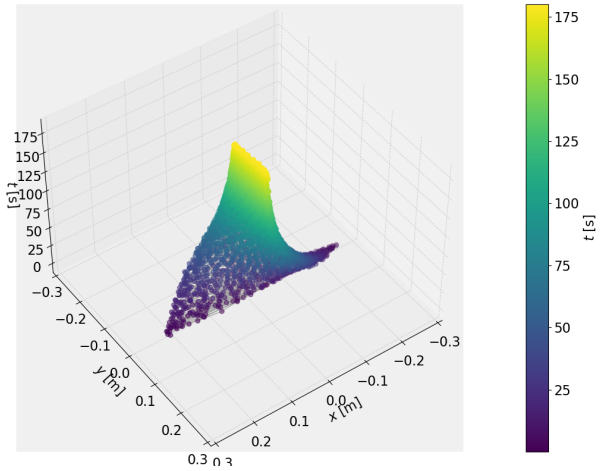


Figura 27: Acquisizione di un oscillazione del pendolo sferico. Si osserva che partendo oscillando in una direzione il pendolo ha una precessione che lo porta su un piano privilegiato.

Stokes, ma questo proposito ottimista si scontra subito con i limiti sperimentali dell'acquisizione. I principali ostacoli sono stati:

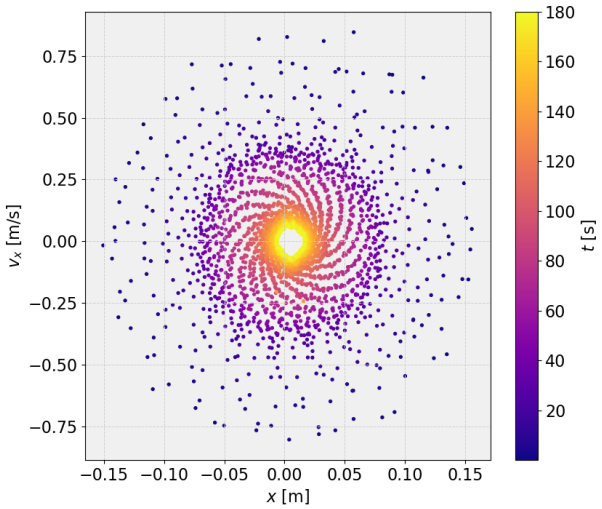


Figura 28: Spezione $x - v_x$ dello spazio delle fasi.

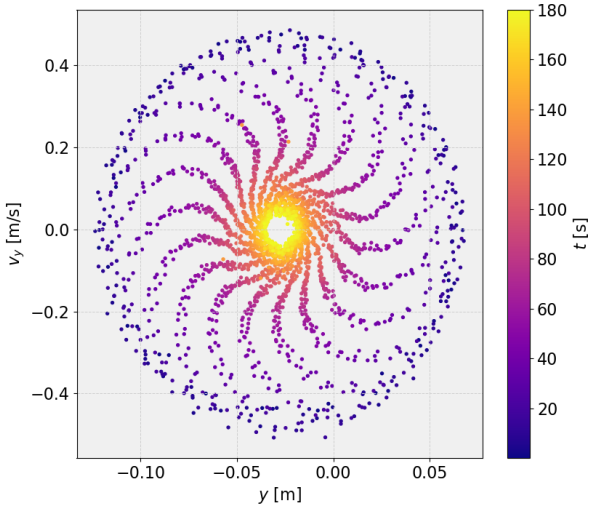


Figura 29: Spezione $y - v_y$ dello spazio delle fasi.

- la sensibilità dello strumento a 1 metro è dell'ordine del mezzo cm, dunque l'idea di usare centinaia di palline di polistirolo ($\approx mm$) risulta non praticabile.
- si è provato ad usare pezzi di spugna dell'ordine del cm, ma una volta inumidite riflettevano l'infrarosso, restituendo una misura nulla di profondità.
- le increspature nell'acqua riflettevano l'infrarosso "accecando" la telecamera, dunque risultava impossibile studiarne i modi superficiali senza contrasto.

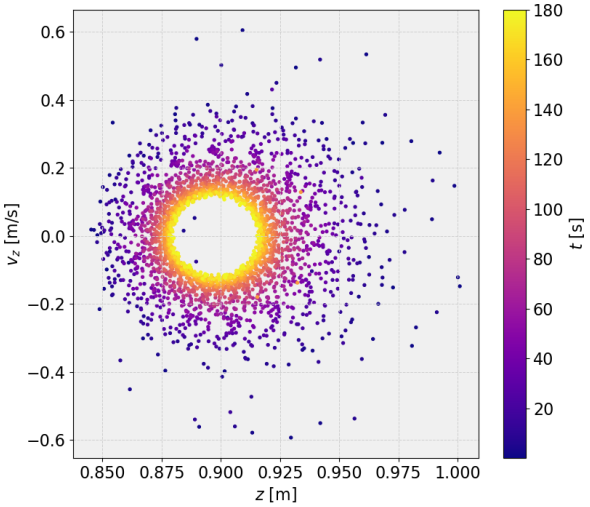


Figura 30: Spezione $z - v_z$ dello spazio delle fasi.

Si è dunque deciso di virare su un semplice esperimento di dinamica, sfruttando il buon tracciamento del sensore per palline più grosse.

Rotazione di una sfera

Per questo primo esperimento si utilizza la telecamera RGB, l'esperimento consiste nel far ruotare una pallina sul pelo dell'acqua. La scelta di farlo in acqua è voluta principalmente per dare stabilità alla rotazione, affinché abbia un unico centro, per non studiare un ulteriore moto traslatorio. La legge del moto è

$$I\ddot{\omega} = -\gamma\dot{\omega}$$

e dunque la sua soluzione è

$$\omega(t) = \omega_0 e^{-\frac{t}{\tau}} \qquad \text{con} \quad \tau = \frac{I}{\gamma}$$

Sulla pallina si disegnano vari puntini e la si mette in moto. Poi si acquisiscono le immagini a 30 fps e si tracciano le coordinate del punto e del centro della sfera a mano. Si calcola la velocità angolare con il limite del rapporto incrementale dell'angolo spazzato e si viene fatto un fit ai minimi quadrati, riportato in figura 31. I residui non sembrano correlati e i dati si adattano abbastanza bene al modello, gli errori sono probabilmente sottostimati. Si è scelto di valutare un errore di 2 pixel in direzione x e y. In generale non si evidenziano motivi particolari per usare il kinect v1 in questo esperimento rispetto ad una qualsiasi fotocamera RGB.

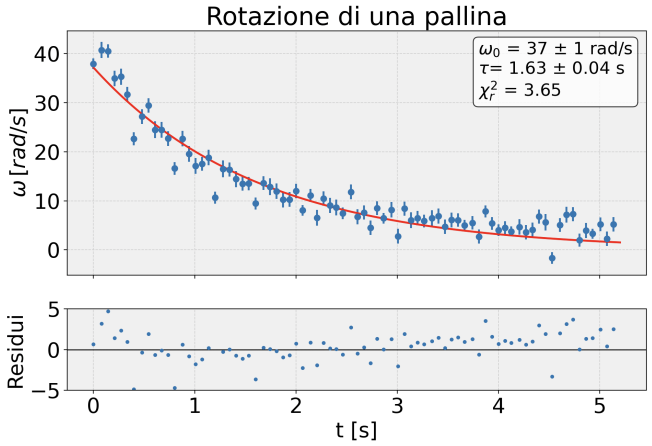


Figura 31: Fit esponenziale.

RICOSTRUZIONE TRIDIMENSIONALE RGB-D: SFM PIPELINE

Si é implementata una Structure from Motion (SfM) pipeline, la quale consente di ricostruire in tre dimensioni un oggetto o una stanza con dimensioni al più di 5 m e risoluzione non migliore di 1 cm.

Lo schema della pipeline è riportato in estrema sintesi in Fig.(32). Si mostrano due acquisizioni effettuate in

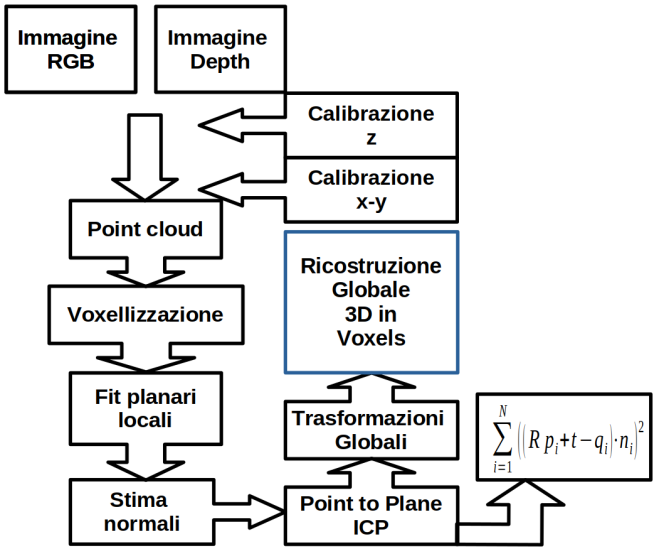


Figura 32: Schema sintetico della pipeline implementata.

Fig.(33) e in Fig.(45).

Come mostrato Kinect v1 si acquisisce:

- un'immagine RGB contenente l'informazione fotometrica della scena;
- una depth map, ovvero una mappa di profondità che associa a ciascun pixel una misura della

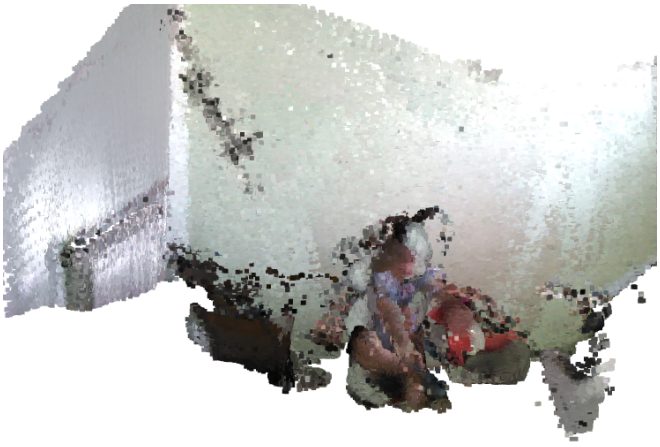


Figura 33: Acquisizione mediante SfM Pipeline di una stanza con soggetto. Sono state utilizzate 200 foto RGB e IR per questa mappatura. Siccome le due telecamere hanno due campi visivi diversi i colori non sono associati perfettamente alle prefondità come si può vedere in Fig.(34). Nome dell'opera: *Biondo tridimensionale.*

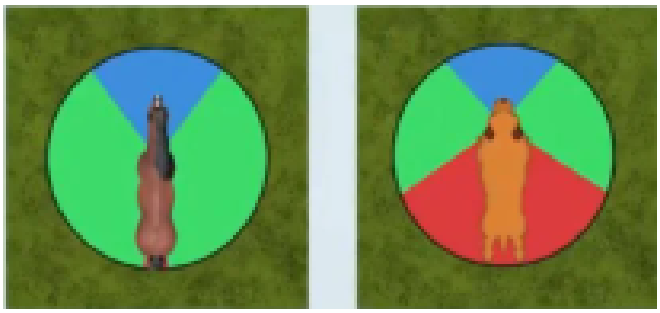


Figura 34: Analogia delle due telecamere: nel nostro processo abbiamo sovrapposto il campo visivo di una mucca (telecamera RGB con campo visivo di $63° \times 50°$) a quello di un cane (telecamera IR con campo visivo di $57° \times 47°$).

distanza tra il punto osservato e la camera.

Questo semplifica significativamente il problema della ricostruzione geometrica della pipeline che in generale può essere fatta ricostruendo la profondità da immagini monoculari mediante triangolazione.

Lo spazio viene rappresentato in *voxels*, unità cubiche di spazio.

Ciascuna coppia di immagini, convertite con i metodi descritti in precedenza, raccoglie un insieme di punti nello spazio (*point cloud*) che, sottocampionandolo, va a popolare i *voxels*.

Per stabilità della acquisizioni si è scelto di prendere come *voxels* cubi di lato 5 cm: si è osservato che prendere troppi voxels, oltre ad essere inefficiente da un punto di vista della memoria necessaria e della potenza di calcolo, va a detrimento della stabilità delle acquisizioni che tendono a essere soggette a divergenze nella pipeline.



Figura 35: Si può vedere una maggiore stabilità dell'acquisizione rispetto che in Fig.(33): questo è dovuto alla presenza di un maggior dettaglio geometrico nella scena che consente alla pipeline di orientarsi molto meglio.

Per ciascun punto della point cloud, così acquisito e tradotto in voxel, viene successivamente stimata la normale locale alla superficie formata coi punti vicini.

L'algoritmo considera i vicini tridimensionali di ciascun punto e approssima localmente la superficie tramite un piano:

$$ax + by + cz + d = 0.$$

Il vettore normale locale risulta quindi:

$$\mathbf{n} = (a, b, c).$$

Successivamente la nuova immagine scattata viene confrontata con le precedenti con l'algoritmo Iterative Closest Point (ICP).

Dato:

- un frame corrente rappresentato dalla point cloud P;
- un frame precedente rappresentato dalla point cloud Q;

ICP ricerca la trasformazione rigida composta da:

- una matrice di rotazione R;
- un vettore di traslazione t;

che minimizza la distanza tra i punti corrispondenti delle due nuvole: ossia l'algoritmo assume che la telecamera si sia spostata di poco e cerca di sovrapporre le stesse forme geometriche che si trovano nelle due immagini. Per questo motivo è bene che il soggetto da mappare non

sia omogeneo. Altri algoritmi associano invece i punti mediante il colore.

Nel caso più semplice, ICP minimizza:

$$\sum_i \|Rp_i + t - q_i\|^2,$$

dove:

- p_i rappresentano i punti della cloud corrente;
- q_i i punti corrispondenti della cloud precedente.

Nell'implementazione utilizzata viene adottata una formulazione *point-to-plane*, nella quale viene minimizzata la distanza rispetto alle normali locali:

$$\sum_i \left((Rp_i + t - q_i) \cdot n_i\right)^2.$$

Questo è il motivo per cui sono stati trovati i piani.

Per ciascun frame è stata così ottenuta una matrice di trasformazione omogenea:

$$T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix},$$

che descrive il moto relativo della camera tra due acquisizioni consecutive.

Le trasformazioni vengono accumulate progressivamente:

$$T_{\text{globale}} = T_1 T_2 T_3 \dots T_n,$$

consentendo di esprimere tutte le point cloud in un unico sistema di riferimento globale. Ogni nuova acquisizione viene quindi trasformata nel riferimento globale e aggiunta alla mappa tridimensionale della scena.

Occorre menzionare che le acquisizioni hanno avuto i seguenti limiti:

- distanza adeguata per il funzionamento corretto del sensore di profondità $(1 - 3\,\text{m})$;
- ambiente illuminato artificialmente per l'adeguato funzionamento del sensore di rpo;
- discretizzazione dello spazio per garantire la stabilità;
- la diversa apertura angolare delle due telecamere ha deformato i colori rispetto alle forme;
- spostamenti estremamente lenti $\sim 1\,\text{cm/s}$.

In fondo all'articolo si riportano i Listings con le implementazioni dell'algoritmo trattato.

CONCLUSIONI

L'esperimento ha raggiunto i suoi obbiettivi: una buona calibrazione che permette la conversione da pixel in metri e l'applicazione di questo nella SfM pipeline. Le principali criticità sono derivate dal diverso punto di vista e dalla distorsioni delle lenti delle due telecamere. Per questa ragione si è effettuata un ulteriore calibrazione per sovrapporre le due immagini trovando i coefficienti di distorsione.

Di contorno si è sperimentato anche il tracciamento di sfere in esperimenti di dinamica. La possibilità di applicare maschere di profondità risulta molto utile per acquisire la traiettoria degli oggetti e velocizza considerevolmente l'analisi delle immagini rispetto ad una normale telecamera RGB. La sensibilità è dell'ordine del cm, dunque permette il tracciamento di oggetti grossi, ma completamente inutile per fare esperimenti più di fino. Inoltre la scarsa velocità di acquisizione (30 fps) può limitare la visione di processi con tempi caratteristici più brevi..

APPENDICE A: ALLINEAMENTO DELLE ACQUISIZIONI RGB E IR

Il problema dell'allineamento

Le due fotocamere, quella RGB e quella IR, quando scattano un immagine allo stesso soggetto, danno due immagini simili ma diverse per i seguenti motivi.

- le telecamere sono fisicamente spaziate da pochi centimetri;
- le telecamere non sono perfettamente allineate;
- le lenti distorcono lo spazio in modo differente.

Il risultato di ciò è che per ogni telecamera separatamente ciascun pixel di un immagine acquisita ha coordinate $[u = f(\phi, \theta), v = g(\phi, \theta)]^T$, dove ϕ e θ sono le coordinate angolari dell'immagine catturata rispetto a un riferimento (identificabile in entrambe le fotocamere con un oggetto molto lontano) e f e g sono funzioni non banali che mappano le coordinate angolari nei pixel corrispondenti.

Sebbene sia possibile provare a fare una calibrazione di questo tipo per entrambe le telecamere, questo esula dagli obiettivi di questa relazione.

La perfetta sovrapposizione di due immagini prese con le due telecamere dunque in genere non è possibile, ma, assunto che il soggetto sia sufficientemente lontano dalle telecamere, può essere fatta in modo ragionevole.

Trasformazione affine

Il modello più semplice è quello di supporre una trasformazione affine che vada a mitigare gli effetti dell'inclinazione tra le due fotocamere e la non coincidenza del centro ottico.

Si considera l'immagine RGB come quella fissa e si vuole trasformare quella IR. Si prendono 20 punti corrispondenti in ciascuna foto come si può vedere in Fig.(36).

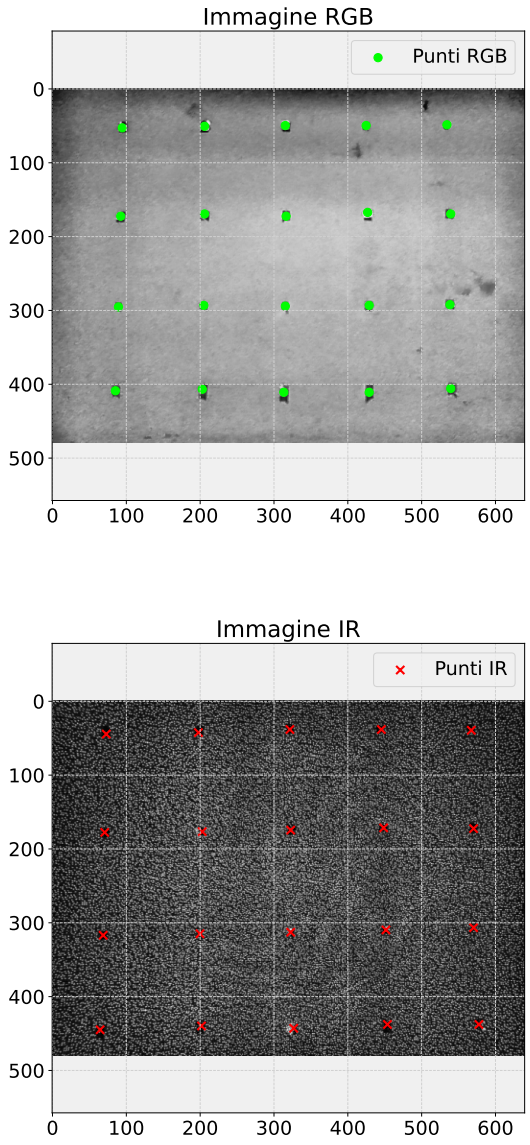


Figura 36: Punti selezionati nelle immagini RGB e IR.

A questo punto utilizzo la seguente funzione di *Open CV2* per trovare la trasformazione tra le due immagini:

Listing 1: Stima della trasformazione affine

```
M, _ = cv2.estimateAffine2D( punti_IR , punti_RGB)
```

Dove M fa la seguente trasformazione in Eq.(10) e rappresentata in Fig(37).

$$\begin{pmatrix} x_{IR}^{\text{allineato}} \\ y_{IR}^{\text{allineato}} \\ 1 \end{pmatrix} = \begin{bmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{pmatrix} x_{IR} \\ y_{IR} \\ 1 \end{pmatrix} \tag{10}$$

dove:

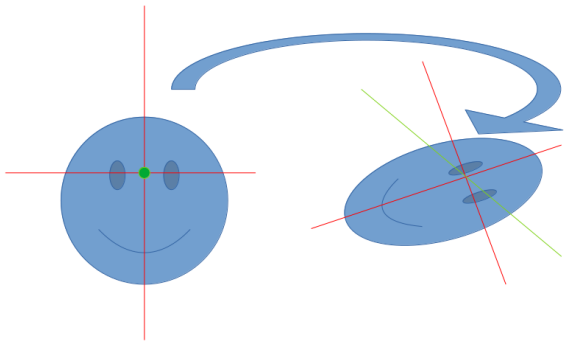


Figura 37: Esempio di trasformazione affine.

- i coefficienti a, b, c, d descrivono rotazioni, shear e scaling nel piano immagine;
- t_x, t_y rappresentano le traslazioni nel piano immagine.

Questo modello di fatto considera la proiezione su un piano di una rotazione nello spazio tridmensionale e una dilatazione lineare. Si è trovato:

$$M = \begin{bmatrix} 0.89 & -0.01 & 30.4 \\ 0.00 & 0.89 & 12.9 \\ 0 & 0 & 1 \end{bmatrix}$$

Infine si applica la trasformazione `warpAffine` di OpenCV all'immagine IR e si sovrappongono, come si può vedere in Fig.(39).

Infine si valutino i risultati con una scena qualunque quale quella in Fig.(40).

Modello di omografia piana

Assumendo che la distanza tra i due sensori sia trascurabile rispetto alla distanza della scena osservata, è possibile utilizzare un modello geometrico più sofisticato basato su un'omografia piana.

Un'omografia rappresenta una trasformazione proiettiva che mette in corrispondenza biunivoca i punti appartenenti a due piani immagine. Dal punto di vista pratico, il

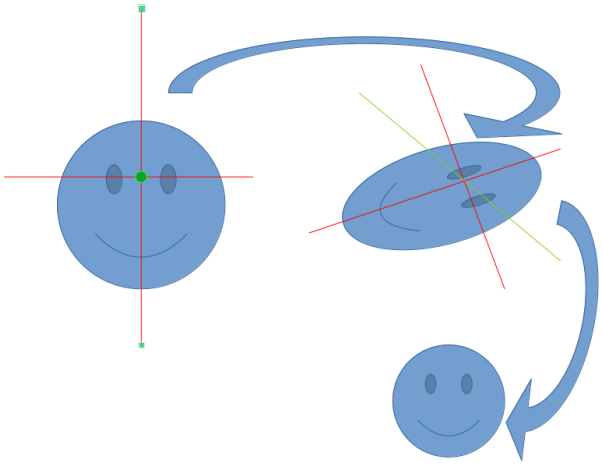


Figura 38: Schema di ciò che viene fatto dall'omografia.

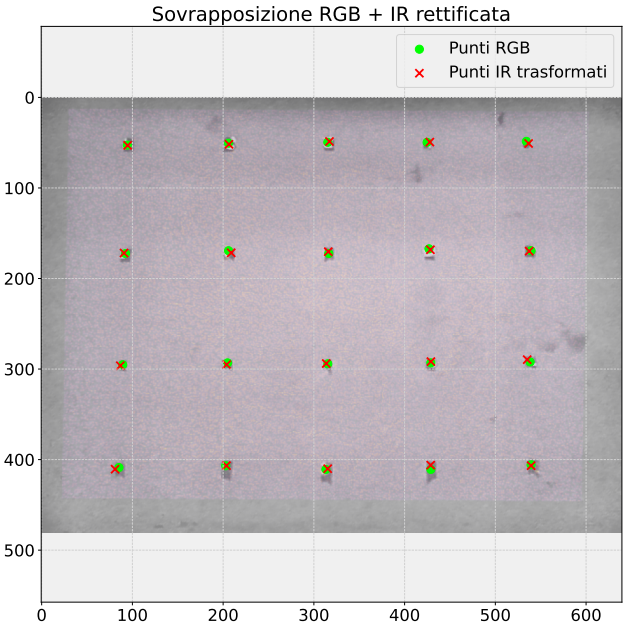


Figura 39: Sovrapposizione con trasformazione affine della parete di calibrazione.

procedimento di selezione dei punti corrispondenti rimane invariato rispetto al caso affine, ma la trasformazione geometrica utilizzata per descrivere la relazione tra le due immagini è differente.

Dato un punto nell'immagine RGB espresso in coordinate omogenee

$$\mathbf{x}_{RGB} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix},$$

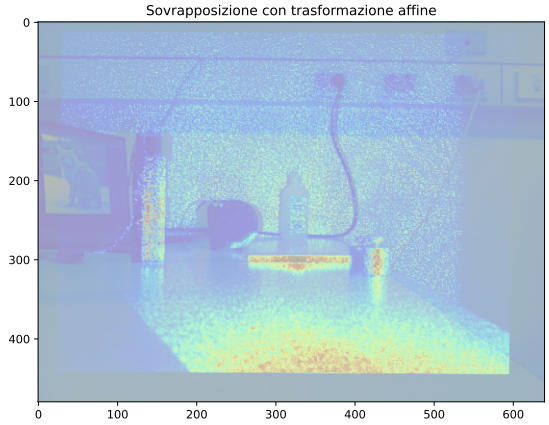


Figura 40: Sovrapposizione con trasformazione affine con una scena qualsiasi.

e il corrispondente punto nell'immagine IR

$$\mathbf{x}_{IR} = \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix},$$

la relazione proiettiva ideale tra i due punti (prima dell'applicazione della distorsione ottica) può essere scritta come:

$$\lambda \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \mathbf{H} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \tag{11}$$

dove:

- λ è un fattore di scala proiettivo;
- $\mathbf{H}$ è la matrice di omografia 3×3.

Poiché un'omografia è definita a meno di un fattore moltiplicativo globale, essa possiede 8 gradi di libertà effettivi. Fissando arbitrariamente il coefficiente $h_{33} = 1$, la matrice assume la forma:

$$\mathbf{H} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix} \tag{12}$$

La trasformazione omografica agisce nello spazio proiettivo bidimensionale. Dopo l'applicazione della matrice $\mathbf{H}$, le coordinate risultano ancora espresse in forma omogenea:

$$(u', v', w').$$

Tuttavia noi vorremmo $w' = 1$, allora imponiamo il fattore λ appositamente, dunque per ottenere le coordinate euclidee finali nel piano immagine è necessario effettuare la divisione prospettica:

$$u = \frac{u'}{w'}, \qquad v = \frac{v'}{w'} \tag{13}$$

La terza coordinata omogenea w' introduce gli effetti prospettici, permettendo di modellare:

- variazioni apparenti di scala;
- punti di fuga;
- inclinazioni prospettiche del piano osservato.

Questi effetti erano completamente ignorati nel primo modello. Sviluppando esplicitamente la trasformazione, si ottiene:

$$u = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + 1}, \qquad v = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + 1}. \tag{14}$$

Dall'ottimizzazione fatta dal codice si ottiene:

$$\mathbf{H} = \begin{bmatrix} 0.86 & 0 & 30.7 \\ 0 & 0.88 & 12.9 \\ 0 & 0 & 1 \end{bmatrix}$$

che confrontata al modello della semplice trasformazione affine rivela che gli effetti prospettici non sono particolarmente dominanti. I valori sono paragonabili e la loro fluttuazione è con ogni probabilità legata alla difficoltà nel cliccare i punti con buona precisione.

Il Modello di Distorsione della Lente (Brown-Conrady Semplificato)

L'omografia proiettiva modella il comportamento di una fotocamera *pinhole* ideale. Tuttavia, i gruppi ottici reali (specialmente nelle lunghezze d'onda dell'infrarosso) introducono aberrazioni geometriche non lineari. Il sistema implementa un sottoinsieme del modello di distorsione di Brown-Conrady, stimando due parametri intrinseci fondamentali:

- **Distorsione Radiale (k_1):** modella l'incurvamento delle linee rette (effetto a barilotto o a cuscinetto) dovuto alla forma sferica delle lenti.
- **Distorsione Tangenziale (p_1):** modella le distorsioni asimmetriche derivanti da un lieve disallineamento fisico (decentramento) tra l'asse ottico della lente e il piano del sensore.

I punti proiettati teorici (u, v) vengono inizialmente normalizzati rispetto alle coordinate del centro ottico (c_x, c_y), ottenendo le coordinate normalizzate $x_{norm} = \frac{u-c_x}{c_x}$ e $y_{norm} = \frac{v-c_y}{c_y}$. Calcolando il quadrato della distanza radiale $r^2 = x_{norm}^2 + y_{norm}^2$, si applica il modello di distorsione.

La deformazione radiale è quantificata come:

$$\begin{cases} \Delta x_{rad} = x_{norm} \cdot (1 + k_1 r^2) \\ \Delta y_{rad} = y_{norm} \cdot (1 + k_1 r^2) \end{cases} \tag{15}$$

La deformazione tangenziale è invece definita dalle equazioni:

$$\begin{cases} \Delta x_{tan} = 2p_1 x_{norm} y_{norm} \\ \Delta y_{tan} = p_1 (r^2 + 2y_{norm}^2) \end{cases} \tag{16}$$

Le coordinate finali distorte e mappate sul sensore IR $(\hat{x}_{ir}, \hat{y}_{ir})$ si ottengono ricombinando le due componenti di disturbo e denormalizzandole rispetto al centro ottico:

$$\begin{cases} \hat{x}_{ir} = (\Delta x_{rad} + \Delta x_{tan}) \cdot c_x + c_x \\ \hat{y}_{ir} = (\Delta y_{rad} + \Delta y_{tan}) \cdot c_y + c_y \end{cases} \tag{17}$$

Nell'ottimizzazione ai minimi quadrati compiuta contemporaneamente all'omografia trattata prima, considerate $k_2 = 0$ e $p_2 = 0$ in quanto si desidera solo provare a sovrapporre le due lenti, si è trovato $k_1 = 0.005$ e $p_1 = -0.0006$. Si rappresentano le distorsioni in Fig.(41). Il risultati si possono leggere in Fig.(42) e in Fig.(44)

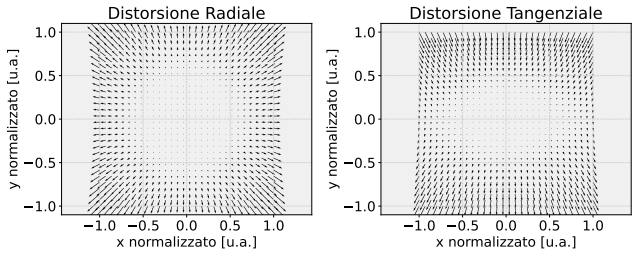


Figura 41: Grafico delle distorsioni.

APPENDICE B: ACQUISIZIONE DELLO SCHELETRO

Cercando su Internet si è trovata una libreria interessante, che riesce a tracciare lo scheletro di una persona alla volta di fronte al Kinect v1. Probabilmente è lo stesso processo che faceva la stessa XBox 360 quando processava i dati acquisiti dal Kinect perché in molti dei giochi bastava effettivamente riconoscere il movimento delle braccia e della gambe.

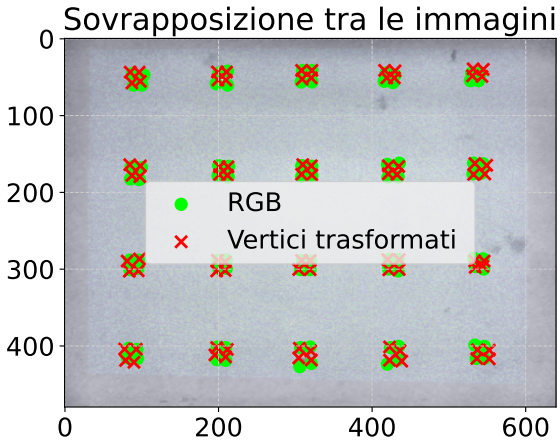


Figura 42: Sovrapposizione dell'immagine IR su quella RGB per la griglia di calibrazione.

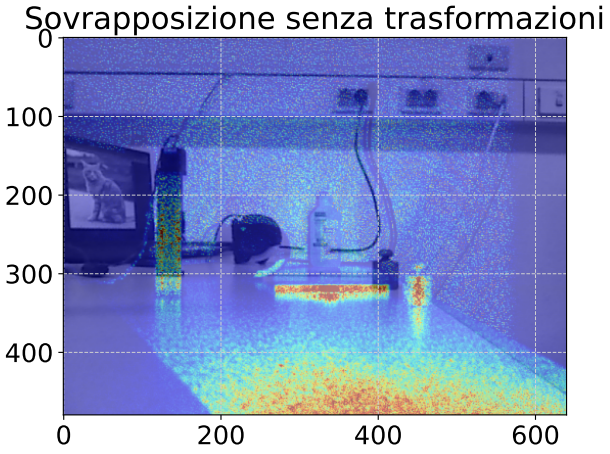


Figura 43: Sovrapposizione di un soggetto qualsiasi senza nessuna trasformazione.

APPENDICE C: CORE DELL'IMPLEMENTAZIONE DELLA PIPELINE

Si riportano di seguito i segmenti di codice, con le rispettive funzioni che essi svolgono, che implementano il core della pipeline.

Listing 2: Creazione point cloud

```
import open3d as o3d
pcd = o3d.geometry.PointCloud()

pcd.points = o3d.utility
             .Vector3dVector(points)
pcd.colors = o3d.utility
             .Vector3dVector(colors)
```

Funzione:

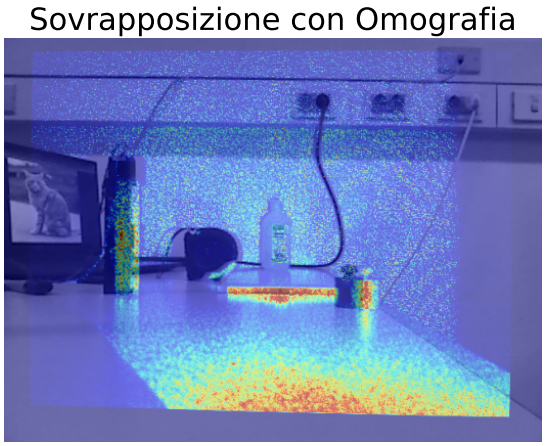


Figura 44: Sovrapposizione di un soggetto qualsiasi dopo l'omografia su l'immagine in infrarosso.

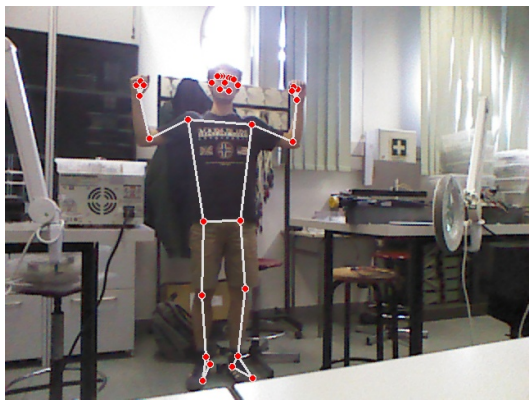


Figura 45: Immagine con riconoscimento dello scheletro: si vede il busto, le gambe, le braccia, mani e piedi. Si intravede qualche dettaglio del volto, come zigomi, arcata sopraccigliare e bocca.

- Creazione della point cloud 3D
- Conversione array NumPy → struttura Open3D
- Associazione coordinate e colori RGB

Listing 3: Voxel downsampling

```
pcd = pcd.voxel_down_sample(size)
```

Funzione:

- Riduzione densità dei punti
- Eliminazione rumore locale
- Accelerazione ICP

Listing 4: Stima delle normali

```
pcd.estimate_normals(
```

```
    search_param=o3d.geometry
    .KDTreeSearchParamHybrid(
        radius=0.08,
        max_nn=30))
```

Funzione:

- Calcolo normali locali
- Necessarie per ICP point-to-plane
- Ricerca vicini tramite KD-tree

Listing 5: Registrazione ICP

```
reg = o3d.pipelines
    .registration
    .registration_icp(
    pcd, prev_pcd, 0.03, np.eye(4),
    o3d.pipelines.registration.
    TransformationEstimationPointToPlane())
T = reg.transformation
```

Funzione:

- Allineamento frame consecutivi
- Stima trasformazione rigida
- Tracking della camera Kinect

Listing 6: Trasformazione globale

```
pcd_global =
    o3d.geometry.PointCloud(pcd)

pcd_global.transform(
    global_transform)
global_pcd += pcd_global
```

Funzione:

- Coordinate locali $\rightarrow$ globali
- Applicazione posa cumulativa